

usuário informar uma **string** e, em seguida, conte **quantas linhas do arquivo /etc/passwd contêm essa string**.

Solicitar ao usuário que digite a string a ser buscada. Contar o número de linhas em /etc/passwd que contenham a string informada. Exibir o resultado de forma clara na tela. Pausar para que o usuário veja o resultado antes de encerrar o script.

```
#!/bin/bash
clear
echo -ne "\n\tContagem de linhas que contenham string em /etc/passwd <ENTER>\n"; read
echo -ne "\tDigite uma string: "; read LIN
NLIN=$( cat /etc/passwd | grep $LIN | wc -l )
echo -ne "\n\tO nº de linhas com a string $LIN de /etc/passwd é $NLIN - <ENTER>\n"; read
echo
```

usuário realizar um **teste de conectividade (ping)** em uma faixa de IPs de uma rede.

Solicitar ao usuário que informe a **rede base** (ex.: 192.168.0). Solicitar ao usuário o **IP inicial** da faixa. Solicitar ao usuário o **IP final** da faixa. Testar cada IP da faixa com **ping** (1 pacote por vez). Registrar os resultados em um arquivo saída.txt. Exibir na tela se cada máquina está **OK** (respondeu ao ping) ou **NÃO OK** (não respondeu). Pausar 1 segundo entre cada teste. Ao final, exibir uma mensagem de término do processo.

```
#!/bin/bash
echo -ne "\n\tDigite a REDE: "; read REDE
echo -ne "\n\tDigite IP1: "; read IP1
echo -ne "\n\tDigite IP2 -FIM: "; read IP2
while [ $IP1 -le $IP2 ]
do
    ping -c 1 $REDE.$IP1 >> saida.txt
    if [ $? -eq 0 ]
    then
        echo -ne "\n\tMaquina: Maq.$IP1 -OK"
    else
        echo -ne "\n\tMaquina: Maq.$IP1 - NÃO OK"
    fi
    IP1=$(( IP1 + 1 ))
    sleep 1
done
echo -ne "\n\tF I M\n"
```

permita criar uma **pseudo-rede**, configurando múltiplos **IPs em interfaces virtuais** de uma máquina Linux.

Exibir mensagens explicativas sobre o programa e sobre o uso de ifconfig. Solicitar ao usuário que informe a **rede base** (ex.: 192.168.0). Solicitar o **primeiro IP** da faixa a ser configurada. Solicitar o **último IP** da faixa a ser configurada. Criar uma interface virtual para cada IP da faixa, usando a nomenclatura eth0:<IP> e

configurando o endereço com ifconfig. Exibir na tela uma mensagem confirmando cada IP configurado. Ao final, exibir uma mensagem de término do processo.

```
#!/bin/bash
echo -ne "\n\tEste programa cria uma 'pseudo rede - <ENTER>"; read
echo -ne "\n\tUsaremos o ifconfig do pacote utils do Linux - <ENTER>"; read
echo -ne "\n\tDigite a REDE: "; read REDE
echo -ne "\n\tDigite o primeiro IP: "; read IP1
echo -ne "\n\tDigite o último IP: "; read IP2
while [ $IP1 -le $IP2 ]
do
    ifconfig eth0:$IP1 $REDE.$IP1
    echo -ne "\n\tRede $REDE.$IP1 configurada \n"
    IP1=$(( IP1 + 1 ))
done
echo -ne "\n\tF I M \n"
```

que demonstre o uso de **funções**.

Definir uma função chamada f1 que:

- Exiba uma mensagem informando que entrou na função.
- Aguarde o usuário pressionar <ENTER>.
- Exiba uma mensagem informando que irá sair da função.

Limpar a tela ao iniciar o programa. Exibir uma mensagem explicando que o programa chamará a função f1 pela primeira vez. Chamar a função f1. Exibir uma mensagem explicando que a função será chamada novamente. Chamar a função f1 novamente. Exibir uma mensagem de despedida e encerrar o script.

```
#!/bin/bash
function f1(){
    echo -ne "\tEstou na função f1 -<ENTER> "; read
    echo -ne "\tVou sair da função f1 -<ENTER> "; read
}
clear
echo -ne "\n\tEste programa chama a função f1 <ENTER> "; read
f1
echo -ne "\n\tO programa chama novamente a função f1 <ENTER> "; read
f1
echo -ne "\n\tO programa irá sair <ENTER> "; read
echo -ne "\n\tTchau!!!\n"
```

demonstre o uso de múltiplas funções e chamadas de funções repetidas.

O script deve:

Definir duas funções:

- f1:
 - Exibe uma mensagem informando que entrou na função f1.
 - Aguarda o usuário pressionar <ENTER>.
 - Exibe uma mensagem informando que irá sair da função f1.

- f2:
- Exibe uma mensagem informando que entrou na função f2.
- Aguarda o usuário pressionar <ENTER>.
- Exibe uma mensagem informando que irá sair da função f2.

Limpar a tela ao iniciar o programa. Exibir mensagens explicando qual função será chamada. Chamar as funções na seguinte ordem: f1, f2, f1 novamente. Exibir uma mensagem de despedida e encerrar o script.

```
#!/bin/bash
function f1(){
    echo -ne "\tEstou na função f1 -<ENTER> ";read
    echo -ne "\tVou sair da função f1 -<ENTER> ";read
}
function f2(){
    echo -ne "\tEstou na função f2 -<ENTER> ";read
    echo -ne "\tVou sair da função f2 -<ENTER> ";read
}
clear
echo -ne "\n\tEste programa chama a função f1 <ENTER> "; read
f1
echo -ne "\n\tEste programa chama a função f2 <ENTER> "; read
f2
echo -ne "\n\tO programa chama novamente a função f1 <ENTER> "; read
f1
echo -ne "\n\tO programa irá sair <ENTER> "; read
echo -ne "\n\tTchau!!!\n"
```

implemente um **menu interativo** com as seguintes opções:

Listar um diretório:

- Solicita ao usuário o caminho de um diretório.
- Verifica se o diretório existe.
- Caso exista, exibe a listagem do diretório.
- Caso não exista, exibe mensagem de erro.

Alterar permissões de um arquivo:

- Solicita ao usuário o caminho absoluto de um arquivo.
- Verifica se o arquivo existe.
- Caso exista, solicita a permissão desejada e aplica usando chmod.
- Exibe a nova permissão do arquivo.

Sair do programa.

Implementar **funções separadas** para cada operação (menu, ler, permissao). Limpar a tela ao exibir o menu ou iniciar uma operação. Ler a opção do usuário e executar a função correspondente. Repetir o menu até que o usuário escolha sair. Exibir mensagens informativas em todas as etapas para facilitar o entendimento do usuário.

```
#!/bin/bash
function menu(){
    clear
    echo -ne "\tMenu\n\n"
    echo -ne "\t1) Lista um diretório\n"
    echo -ne "\t2) Altera as permissões de um arquivo\n"
    echo -ne "\t3) Sai\n"
    echo -ne "\n\n\tDigite a opção: "
}
function ler(){
    clear; echo -ne "\n\tDigite um diretório: "; read LIN
    if [ -d $LIN ]
    then
        echo -ne "\n\tDiretório $LIN existe\n"
    else
        echo -ne "\n\tDiretório $LIN NÃO existe <ENTER>\n";read; return 0
    fi
    ls $LIN
    echo -ne "\n\tAcima está a listagem de $LIN - <ENTER>\n";read
}
function permissao(){
    clear
    echo -ne "\n\tDigite o cam. absoluto do arq: "; read ARQ
    if [ ! -f $ARQ ]
    then
        echo -ne "\n\tNão existe arq. $ARQ <ENTER>";read; return 0
    fi
    echo -ne "\n\tDigite a permissão de um arquivo: "; read PER
    chmod $PER $ARQ
    ls -l $ARQ
    echo -ne "\n\tPermissão do arquivo $ARQ <ENTER>\n";read
}
op=0
while [ $op -ne 3 ]
do
    menu
    read op
    echo -ne "\n\tOpção digitada $op <ENTER>\n"; read
    if [ $op -eq 1 ]
    then
        ler
    fi
    if [ $op -eq 2 ]
    then
        permissao
    fi
done
echo -ne "\n\tAté logo! \n"
```

realiza a **contagem de arquivos em um diretório** informado pelo usuário.

Exibir uma mensagem explicando a função do programa. Solicitar ao usuário que digite o caminho de um **diretório existente**. Verificar se o diretório existe; caso não exista, exibir mensagem de erro e encerrar o programa. Contar todos os arquivos presentes no diretório. Listar os arquivos encontrados, exibindo um número sequencial antes de cada arquivo. Ao final, exibir a quantidade total de arquivos no diretório.

```
#!/bin/bash
clear
echo -ne "\n\tContagem de arquivo em um diretório - <ENTER>\n"; read
echo -ne "\n\tDigite um diretório existente: "; read DIR
if [ ! -d $DIR ]
then
    echo -ne "\n\tDiretório $DIR Não existe - <ENTER>\n"; read; exit
fi
CONT=0
for VAR in $(ls $DIR)
do
    CONT=$(( CONT + 1 ))
    echo -ne "\n\t$CONT) VAR = $VAR"
done
echo
echo -ne "\n\tNo diretório $DIR, existem $CONT arquivos <ENTER>\n"; read
```

Exiba uma mensagem informando que irá listar o conteúdo do diretório /bin.

Aguarde que o usuário pressione <ENTER> antes de executar a listagem.

Liste todos os arquivos e diretórios presentes em /bin.

Exiba uma mensagem informando que a listagem foi concluída e aguarde <ENTER>.

Exiba uma mensagem de despedida e aguarde <ENTER> antes de encerrar o script.

```
#!/bin/bash
echo "Este programa lista o diretório /bin - <ENTER>"
read
ls /bin
echo "Acima está a listagem de /bin - <ENTER>"
read
echo "Até logo! <ENTER>"
read
```

Solicite ao usuário que digite o **caminho absoluto de um diretório existente**.

Exiba uma mensagem informando que irá listar o conteúdo do diretório informado.

Aguarde que o usuário pressione <ENTER> antes de executar a listagem.

Liste todos os arquivos e subdiretórios presentes no diretório informado.

Exiba uma mensagem indicando que a listagem foi concluída e aguarde <ENTER>.

Exiba uma mensagem de despedida e aguarde <ENTER> antes de encerrar o script.

```
#!/bin/bash
echo "Digite o caminho absoluto de um diretório existente"
read DIR
echo "Este programa lista o diretório $DIR - <ENTER>"
read
ls $DIR
echo "Acima está a listagem de $DIR - <ENTER>"
read
echo "Até logo! <ENTER>"
read
```

Solicite ao usuário que digite seu nome.

Solicite ao usuário que digite sua idade.

Verifique se o usuário é maior de idade (idade $\geq$ 18 anos).

- Se for maior de idade, exiba uma mensagem informando que ele pode assistir ao filme, incluindo seu nome e idade.

- Se for menor de idade, exiba uma mensagem informando que ele não pode assistir ao filme, incluindo seu nome e idade.

Ao final, exiba uma mensagem de despedida e aguarde <ENTER> antes de encerrar o script.

```
#!/bin/bash
echo "Digite o nome de um usuário"
read NOME
echo "Digite a idade do usuário"
read IDADE
if [ $IDADE -ge 18 ]
then
    echo "Olá $NOME, você pode assistir ao filme!"
    echo "Isso porque você tem a idade de $IDADE anos - você é maior de idade!"
else
    echo "Olá $NOME, você NÃO pode assistir ao filme!"
    echo "Isso porque você tem a idade de $IDADE anos - você é MENOR de idade!"
fi
echo
echo "Até logo - <ENTER>"
read
```

realize as seguintes operações: Solicite ao usuário que digite o caminho absoluto de um diretório.

Verifique se o diretório existe:

- Caso exista, exiba mensagens de confirmação e continue a execução.
- Caso não exista, exiba mensagens de erro e encerre o programa.

Se o diretório existir, exiba uma mensagem informando que será realizada a listagem do diretório e aguarde <ENTER>.

Liste todos os arquivos e subdiretórios do diretório informado.

Exiba uma mensagem indicando que a listagem foi concluída e aguarde <ENTER>.

Exiba uma mensagem de despedida e aguarde <ENTER> antes de encerrar o script.

```
#!/bin/bash
echo "Digite o caminho absoluto de um diretório existente"
read DIR
if [ -d $DIR ]
then
    echo "Diretório $DIR existe!"
    echo "Você digitou corretamente! <ENTER>"
    read
else
    echo "Diretório $DIR NÃO existe!"
    echo "Você deve ter digitado algo errado!"
    echo "Vamos abandonar o programa! <ENTER>"
    read
    exit
fi
echo "Este programa lista o diretório $DIR - <ENTER>"
read
ls $DIR
echo "Acima está a listagem de $DIR - <ENTER>"
read
echo "Até logo! <ENTER>"
read
```

Verificar se o [Linux](#) identificou o disco com o comando:

fdisk -l

2. Após verificar, efetuar o seguinte comando com o fdisk para dar início a criação da partição:

fdisk /dev/sdb1

(mude /dev/sdb1 de acordo com sua configuração de discos)

3. Após executar o comando acima o mesmo irá mostrar várias opções que o fdisk possui, utilizarem a seguintes neste exemplo:

- opção **p** --> mostra qual disco será criada a partição. Sempre utilize essa opção para ter certeza em qual disco você se encontra.
- opção **n** --> utilizado para criar uma nova partição, logo em seguida aparecerão várias perguntas se deseja que a partição seja primária ou estendida etc.

5. Formataremos a partição com o tipo de sistema de arquivo desejado:

mkfs -t ext4 /dev/sdb1

6. Agora execute o comando para montar a partição:

mount /dev/sdb1 /opt/

7. Pronto, foi feita a criação de uma partição e a sua montagem, se tudo ocorrer bem a partição será exibida com o comando:

df -h